PROJECT LAB REPORT

NAME – **ANIKET ROY**

ENROLLMENT NO – **12024052017032**

STREAM – **CSE (IOT CSBT)**

SEC – **A**

ROLL – **15**

SUBJECT –

DATA STRUCTURES AND ALGORITHM LAB

SUB CODE – **PCCCS391**

TOPIC – **TRAIN RESERVATION SYSTEM**

INDEX

Sl no	topic	Page no.
1	PROBLEM STATEMENT	1
2	ALGORITHM	2
3	CODE	3-8
4	OUTPUT	8-10
5	DISCUSSION	10

PROBLEM STATEMENT

In today's world, railway reservation systems are essential for managing thousands of passengers efficiently. Manual booking can lead to delays, overbooking, and difficulties in tracking cancellations and waiting lists.

The Train Reservation System aims to simulate a simplified model of railway ticket booking using Data Structures and Algorithms concepts like Arrays, Linked Lists, and Queues.

This project manages seat booking, waiting list maintenance, and cancellation operations in a logical, automated manner. When all seats are booked, new passengers are placed in a waiting queue. If a seat is canceled, it is automatically assigned to the first passenger in the waiting queue.

Objectives:

- **Build a menu-driven Train Reservation System in C.**
- **Use an Array to represent booked seats.**
- **Use a Queue to manage the waiting list.**
- **Use a Linked List to store passenger details dynamically.**
- **Implement booking, cancellation, status display, and waiting list update.**
- **Provide efficient and modular code structure following DSA principles.**

Algorithm

1. **Initialize all data structures — an array to store seat status, a linked list for passenger records, and a queue for waiting passengers.**
2. **Check for seat availability when a booking request is made. If a seat is available, assign it to the passenger, store the details in the linked list, and mark the seat as booked.**
3. **If no seats are available, insert the passenger into the waiting queue, maintaining the order of requests.**
4. **When a cancellation request is received, remove the passenger's record from the linked list and mark the corresponding seat as empty.**
5. **If the waiting queue is not empty, remove the first passenger from the queue and assign the newly freed seat to them, updating all related details.**
6. **Display the current list of booked passengers and waiting passengers whenever requested, ensuring all data structures remain synchronized throughout program execution.**

CODE

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#define MAX_SEATS 5

#define MAX_WAITING 5

typedef struct Passenger {

    char name[50];

    int age;

    int seatNo;

    char trainNo[10];

    struct Passenger *next;

} Passenger;

typedef struct Queue {

    char name[50];

    int age;

    char trainNo[10];

} Queue;

Passenger *head = NULL;

Queue waiting[MAX_WAITING];

int front = -1, rear = -1;

int seats[MAX_SEATS] = {0};

void enqueueWaiting(char name[], int age, char trainNo[]) {

    if (rear == MAX_WAITING - 1) {

        printf("\nWaiting list is full!\n");

        return;

    }

    if (front == -1) front = 0;

    rear++;

    strcpy(waiting[rear].name, name);
```

```
    waiting[rear].age = age;
    strcpy(waiting[rear].trainNo, trainNo);
}
int isWaitingEmpty() {
    return (front == -1 || front > rear);
}
Queue dequeueWaiting() {
    Queue q = {"", 0, ""};
    if (isWaitingEmpty()) return q;
    q = waiting[front];
    front++;
    if (front > rear) {
        front = rear = -1;
    }
    return q;
}
void bookTicket() {
    char name[50], trainNo[10];
    int age;
    printf("\nEnter passenger name: ");
    scanf("%s", name);
    printf("Enter age: ");
    scanf("%d", &age);
    printf("Enter train number: ");
    scanf("%s", trainNo);
    int seatNo = -1;
    for (int i = 0; i < MAX_SEATS; i++) {
        if (seats[i] == 0) {
            seatNo = i + 1;
            seats[i] = 1;
```

```
            break;
        }}
        if (seatNo != -1) {
        Passenger *newP = (Passenger*)malloc(sizeof(Passenger));
        strcpy(newP->name, name);
        newP->age = age;
        strcpy(newP->trainNo, trainNo);
        newP->seatNo = seatNo;
        newP->next = NULL;
         if (head == NULL)
            head = newP;
        else {
            Passenger *temp = head;
            while (temp->next != NULL)
                temp = temp->next;
            temp->next = newP;
        }
     printf("\nTicket booked successfully! Seat No: %d\n", seatNo);
    } else {
        printf("\nAll seats full. Added to waiting list.\n");
        enqueueWaiting(name, age, trainNo);
    }}
void cancelTicket() {
    if (head == NULL) {
        printf("\nNo bookings yet!\n");
        return;
    }
char name[50];
    printf("\nEnter passenger name to cancel: ");
    scanf("%s", name);
```

```
Passenger *temp = head, *prev = NULL;

while (temp != NULL && strcmp(temp->name, name) != 0) {

    prev = temp;

    temp = temp->next;

}

    if (temp == NULL) {

    printf("\nNo passenger found with that name.\n");

    return;

}

int canceledSeat = temp->seatNo;

if (prev == NULL)

    head = temp->next;

else

    prev->next = temp->next;

   free(temp);

seats[canceledSeat - 1] = 0;

printf("\nTicket canceled. Seat No %d is now free.\n", canceledSeat);


if (!isWaitingEmpty()) {

    Queue next = dequeueWaiting();

    Passenger *newP = (Passenger*)malloc(sizeof(Passenger));

    strcpy(newP->name, next.name);

    newP->age = next.age;

    strcpy(newP->trainNo, next.trainNo);

    newP->seatNo = canceledSeat;

    newP->next = NULL;

     if (head == NULL)

        head = newP;

    else {
```

```
            Passenger *t = head;
            while (t->next != NULL)
                t = t->next;
            t->next = newP;
        }
        seats[canceledSeat - 1] = 1;
        printf("Waiting passenger %s assigned to seat %d.\n", next.name, canceledSeat);
    }}
void displayStatus() {
    printf("\n--- Booked Passengers ---\n");
    if (head == NULL) {
        printf("No bookings yet.\n");
    } else {
        Passenger *temp = head;
        while (temp != NULL) {
            printf("Seat %d: %s (Age %d, Train %s)\n",
                    temp->seatNo, temp->name, temp->age, temp->trainNo);
            temp = temp->next;
        }}
printf("\n--- Waiting List ---\n");
    if (isWaitingEmpty()) {
        printf("No passengers in waiting list.\n");
    } else {
        for (int i = front; i <= rear; i++) {
            printf("%d. %s (Age %d, Train %s)\n",
                    i - front + 1, waiting[i].name, waiting[i].age, waiting[i].trainNo);
        }}}
int main() {
    int choice;
    while (1) {
```

```
    printf("\n1. Book Ticket\n2. Cancel Ticket\n3. View Status\n4. Exit\nEnter your choice: ");
    scanf("%d", &choice);
    switch (choice) {
        case 1: bookTicket(); break;
        case 2: cancelTicket(); break;
        case 3: displayStatus(); break;
        case 4: printf("\nExiting system...\n"); return 0;
        default: printf("\nInvalid choice! Try again.\n");
    }}
return 0;
}
```

OUTPUT

1. Book Ticket

2. Cancel Ticket

3. View Status

4. Exit

Enter your choice: 1

Enter passenger name: Rahul

Enter age: 25

Enter train number: 12345

Ticket booked successfully! Seat No: 1

Enter your choice: 1

Enter passenger name: Neha

Enter age: 28

Enter train number: 12345

Ticket booked successfully! Seat No: 2

Enter your choice: 1

Enter passenger name: Amit

Enter age: 30

Enter train number: 12345

Ticket booked successfully! Seat No: 3

Enter your choice: 1

Enter passenger name: Arjun

Enter age: 22

Enter train number: 12345

Ticket booked successfully! Seat No: 4

Enter your choice: 1

Enter passenger name: Sima

Enter age: 27

Enter train number: 12345

Ticket booked successfully! Seat No: 5

Enter your choice: 1

Enter passenger name: Riya

Enter age: 24

Enter train number: 12345

All seats full. Added to waiting list.

Enter your choice: 3

--- Booked Passengers ---

Seat 1: Rahul (Age 25, Train 12345)

Seat 2: Neha (Age 28, Train 12345)

Seat 3: Amit (Age 30, Train 12345)

Seat 4: Arjun (Age 22, Train 12345)

Seat 5: Sima (Age 27, Train 12345)

--- Waiting List ---

1. Riya (Age 24, Train 12345)

Enter your choice: 2

Enter passenger name to cancel: Sima

Ticket canceled. Seat No 5 is now free.

Waiting passenger Riya assigned to seat 5.

Enter your choice: 3

--- Booked Passengers ---

Seat 1: Rahul (Age 25, Train 12345)

Seat 2: Neha (Age 28, Train 12345)

Seat 3: Amit (Age 30, Train 12345)

Seat 4: Arjun (Age 22, Train 12345)

Seat 5: Riya (Age 24, Train 12345)

--- Waiting List ---

No passengers in waiting list.

DISCUSSION

The Train Reservation System efficiently manages booking, cancellation, and waiting lists using fundamental data structures.
Arrays are used to track seat availability quickly, ensuring constant-time access.
Linked lists store passenger details dynamically, allowing easy insertion and deletion.
Queues handle waiting passengers in a fair first-come-first-served order.
The system automatically assigns canceled seats to the next waiting passenger.
This structure provides clear logical flow and practical use of DSA concepts.
The implementation reduces manual effort and improves reservation efficiency.
Overall, the project demonstrates how data structures can solve real-world allocation problems effectively.